Seeding metaheuristics with learned and heuristic solution pools for the Interval Job Shop Scheduling Problem: an anytime study of when initialisation helps

Hernán Díaz Rodríguez[a]

[a] *Department of Computer Science, University of Oviedo, Spain*

Abstract

Practitioners routinely assume that seeding a metaheuristic's initial population with good solutions improves its results. We test it on the Job Shop Scheduling Problem with interval processing times, using four published solvers spanning the weak-to-strong axis, four seed generators — a deep reinforcement-learning policy and a ladder of randomised constructives — and a mixed pool, over 61 interval Taillard instances and 30 runs per cell (51,240 runs). A pilot study first identifies three practices that manufacture positive seeding results: stagnation-triggered stopping, single-budget comparison, and self-referential speed metrics.

With the instance as the unit of analysis, the sign of the effect is not predicted by solver strength. A mixed pool improves the weakest solver by 1.14 percentage points of deviation from published lower bounds (95% CI $[-1.36, -0.93]$), leaves a memetic ABC unchanged, and does not harm a genetic algorithm, although every single-generator pool does. On the strongest solver the improvement is small, confined to one size class, and absent once the analysis is restricted to instances whose control actually converged. What matters more is the budget: at a tenth of ours, seeding cuts the excess over the lower bound on all four solvers, but by margins spanning an order of magnitude — 11% on the genetic algorithm against 40% on the memetic ABC — and the two largest gains accrue to the two *strongest* solvers, reversing the ordering at the budget limit.

Two observations constrain the mechanism. The mixed pool draws on the

Email address: herdiaz21@uniovi.es (Hernán Díaz Rodríguez)

same solutions as the single-generator arms and starts from a worse average population, yet converges better, implicating composition rather than seed quality. And premature convergence, the usual explanation for seeding failures, is contradicted: the harmed genetic algorithm ends *more* dispersed than its control, not less.

Keywords: Job shop scheduling, interval uncertainty, population seeding, metaheuristics, reinforcement learning, anytime behaviour

---

1. Introduction

Real-world scheduling rarely enjoys exact knowledge of task durations. When only lower and upper bounds are available, the Job Shop Scheduling Problem with interval processing times (IJSP) models each duration as a closed interval and propagates interval arithmetic through the schedule [1, 2]. The IJSP has accumulated a substantial algorithmic literature over the last years, including genetic algorithms [3], elitist artificial bee colony variants [4, 5], fast memetic ABC hybrids [6], and, most recently, tabu-search hybrids built on extreme-critical-path neighbourhoods that constitute the current state of the art [7].

In parallel, learning-based methods have begun to produce fast constructive policies for shop scheduling [8, 9]. A deep reinforcement-learning (RL) dispatching policy for the IJSP can emit thousands of feasible, above-average solutions per instance in seconds, raising an obvious question: should we seed the initial populations of existing metaheuristics with such solutions? Folk wisdom answers yes, and heuristic seeding of evolutionary algorithms has a long history [10, 11]. Yet the empirical evidence behind that wisdom is weaker than it appears. Seeding studies typically compare final objective values under a fixed stopping rule, and our pilot experiments show that this practice is fragile in three specific ways.

First, *stagnation-triggered stopping*: under the customary "N iterations without improvement" rule, a well-seeded population that the solver cannot immediately improve triggers the rule at the floor value, so seeded runs are cut short

while unseeded runs search for an order of magnitude longer — the comparison silently penalises the seeds. Second, *budget-dependent gains*: under a fixed budget chosen without a convergence check, seeded arms of weak solvers appear to win; with a five-fold larger budget the unseeded control keeps improving, overtakes the stagnating seeded population, and the sign of the effect *reverses*. Third, *self-referential speed*: convergence speed is reported as the time to the arm's own plateau, so an arm that stagnates early on a worse value is credited with an acceleration. Any of these three artefacts can manufacture a positive seeding result where none exists.

This paper makes three contributions.

1. **A pre-specified anytime methodology** for evaluating population seeding: convergence-checked final quality; two complementary time-to-target metrics (time to the arm's own plateau, and time to the unseeded control's final quality) reported with the proportion of instances on which the target is reached; and paired runs with per-cell budgets calibrated on the control.
2. **A portfolio study across the weak-to-strong axis** — GA [3], elitist ABC [4], memetic fEABC-LS [6], and the tabu hybrid TS-N2 [7] — showing that the sign of the seeding effect is *not* predicted by solver strength: the weakest solver gains, the strongest gains on the largest instances, one is unaffected, and one is degraded. The sign tracks whether the solver's variation operators can consolidate a structurally heterogeneous population, and the magnitude tracks the ratio of instance size to search budget; both are associations our design supports rather than causal effects it isolates.
3. **Learned versus heuristic seeds**: the RL policy is compared, under identical injection protocols, against a quality ladder of randomised constructives (machine-operations-remaining, randomised Giffler–Thompson, and a tuned genetic-programming rule), plus a mixed pool assembled from the same solutions. The mixed pool starts from a *worse* average population than its single-generator counterparts and still converges better,

which points to pool composition rather than seed quality as the operative variable.

Our results do not support the assumption we set out to test. Over 51,240 runs, a mixed seed pool improves the weakest solver on 35 of 61 instances without losing on any, leaves a memetic ABC statistically unchanged, degrades a genetic algorithm, and improves a strong tabu hybrid only on the largest instance class — where, however, every generator we tried beats the unseeded control. The sign of the effect is therefore not predicted by solver quality. What does predict it is whether the solver's variation operators can exploit a structurally heterogeneous population, and what predicts its magnitude is the ratio of instance size to search budget: at one tenth of the budget seeding reduces the excess over the published lower bounds by 11% to 40% on all four solvers, including the one it harms at convergence.

The paper is organised as follows. Section 2 reviews related work. Section 3 defines the IJSP and the solver portfolio. Section 4 describes the seed generators, the pool audit and the injection mechanism. Section 5 reports the pilot study whose failure modes motivate the design of Section 6. Section 7 presents the results, Section 8 discusses their implications, and Section 9 concludes.

2. Related work

Metaheuristics for the IJSP.. Lei introduced population-based neighbourhood search for interval makespan minimisation [1] and a GA for interval due-date objectives [2]. A GA tailored to interval uncertainty with an insertion schedule-generation scheme was proposed in [3], later outperformed by elitist ABC variants [4], a seasonal elitist ABC [5], and the fast elitist ABC with hill-climbing local search (fEABC-LS) [6]. The most recent entry couples the fEABC framework with a best-improvement tabu search over block-boundary neighbourhoods defined on both extreme scenario graphs [7]. Our portfolio comprises exactly these published solvers, replicated from their original configurations (Section 3).

Population initialisation and seeding.. Initialisation strategies for evolutionary algorithms have been surveyed in [10]; injecting known solutions has been studied as case-injected genetic search [11], and constructive-heuristic seeding is standard practice in scheduling applications. What distinguishes the present study is not the mechanism — ours is deliberately the simplest possible: replace k random individuals by pool solutions at initialisation — but the evaluation methodology: convergence checks, anytime metrics, and paired statistics with multiple-comparison control.

Learning to construct schedules.. Deep RL dispatchers for the deterministic JSP [8] and the broader learn-to-optimise programme [9] produce fast constructive policies whose natural integration point with mature metaheuristics is precisely initialisation. We treat the learned policy as one seed generator among several, which allows the question *is a learned constructive better than a tuned heuristic one as a seed source?* to be answered under identical injection protocols.

3. The IJSP and the solver portfolio

3.1. Problem definition

An IJSP instance consists of n jobs, each a fixed sequence of m operations, one per machine. The processing time of operation o is a closed interval $p_o = [p_o^-, p_o^+]$. Interval addition and component-wise maximum extend the classical recursion for starting and completion times: for a feasible processing order σ, the starting time of o is $s_o = \max(c_{PJ(o)}, c_{PM(o)})$ with $c_o = s_o + p_o$, where PJ and PM denote job and machine predecessors and the maximum is taken component-wise [1, 3]. The makespan $C_{\max} = [C_{\max}^-, C_{\max}^+]$ is the component-wise maximum of job completion times; it is exact, because the makespan is monotone in the durations, so the all-lower and all-upper configurations realise its bounds. Comparing schedules requires an interval ranking operator; we use the expected-value ranking of [12], i.e. the interval midpoint, which we write $\mathbb{E}[C_{\max}]$ following the convention of this literature; no probability distribution over the interval is assumed, and the quantity is a ranking criterion rather than

a mathematical expectation. It ranks solutions for the evolutionary solvers, following their original papers, while LEX2 (lexicographic-by-upper-bound) is used inside TS-N2, following [7]; all cross-arm comparisons are made on $\mathbb{E}[C_{\max}]$ within a single solver, so the ranking choice never confounds the seeding effect.

Instances are the standard interval Taillard benchmark: each crisp duration p_o becomes $[p_o - \delta,\, p_o + \delta]$ with δ drawn uniformly from $[0,\, 0.15\, p_o]$ [3, 13].

3.2. The solver portfolio

All four solvers share the permutation-with-repetition encoding [14], the insertion schedule-generation scheme (active schedules) for decoding, population size 250, and the component-wise interval evaluation described above. Table 1 summarises their published configurations, whose parameter settings we replicate verbatim.

3.3. Replication check

Before any seeding experiment we replicated each solver against its published results on the twelve classical interval instances. The GA and fEABC replications match the published average relative errors within $\sim$1 percentage point on all twelve instances (fEABC matches or slightly improves on every one); fEABC-LS reproduces the published ranking (best of its family). Our ABCE3 replication runs systematically $\sim$1.5–2.5 pp above the published averages; we were unable to close this gap from the published description and disclose it here.

We take the published parameter settings verbatim, but that is a statement about the configuration, not a claim that our implementation is behaviourally identical to the authors' own. The ABCE3 gap above shows it need not be, and the local-search footnote to Table 1 shows where the two can diverge while the parameter file reads the same. Neither affects any conclusion of this paper, because every seeding comparison is within-solver: a seeded arm against the unseeded control of the same implementation, under the same budget, on the same instances.

Table 1: Solver portfolio (published configurations). All use population 250, permutation-with-repetition encoding and insertion SGS decoding.

	GA [3]	ABCE3 [4]	fEABC-LS [6]	TS-N2 [7]
Frame	generational GA	elitist ABC	fast elitist ABC	fast elitist ABC
Crossover	JOX, p_c=1.0	GOX, p=1.0	JOX	JOX
Mutation/move	insertion, p_m=0.15	swap, p=1.0	swap	insertion
Elite / trials	—	B=50, $\mathrm{NT}_{\max}$=20	40 / 15	50 / 24
Local search	none	none	HC (first impr.), N_1	tabu (best impr.), N_2, LEX2
LS control	—	—	best of each trial pair[†]	100%, $B_{\max}$=20, $T_{\max}$=2s

[†] The published configuration sets the local-search target to 75% of the population, and that is the value we ran. Its effect in this implementation is narrower than the figure suggests, and we state what the code does rather than what the parameter names: local search is invoked on the two-member candidate population formed by each employed-bee trial, and $\lfloor 0.75 \times 2 \rfloor - 1 = 0$ leaves the random quota empty, so only the better of the two candidates is improved (`ArtificialBeeColonyPSO.cpp:559-574,705,727`). Every arm of this solver runs the identical mechanism, so within-solver comparisons are unaffected; the qualification bears on how the solver should be described, and on the replication gap reported below.

4. Seed pools and injection mechanism

4.1. Generators and pools

For every instance we use pools of 1,024 i.i.d. solutions per generator, each stored as a job-permutation plus its makespan interval under a semi-active decoder. Four generators span a quality ladder:

- **MOR**-ε: most-operations-remaining dispatching with $\varepsilon = 0.1$ random tie-perturbation (weak baseline);
- **GT**-ε: randomised Giffler–Thompson with ε-noise inside the conflict set (active schedules; strong baseline);

- **GP-ε**: a genetic-programming priority rule, evolved and irace-tuned offline, with GRASP-style ε-noise (strongest heuristic baseline);
- **v2**: a deep-RL dispatching policy (deep-sets architecture, three independent training checkpoints), trained only on Taillard instances TA11–TA14 and developed/tuned on TA15–TA20.

Across the 71-instance pool collection the ladder is clean: mean relative errors of 51.1% (MOR) $>$ 30.3% (GT) $>$ 21.4% (GP) $\approx$ 21.2% (v2), with v2 exhibiting the highest structural diversity of the four (0.95 vs. 0.92 for GP on a normalised disjunctive-order distance) and GP slightly stronger on the largest size classes. A fifth, mixed pool (equal parts v2/GT/GP; MOR is deliberately excluded, being the weakest generator) is used by the *mix* arm. All five pools — the four generators and the mixture — contain 1,024 solutions per instance, so no arm is sampled more richly than another.

The mixed pool interleaves the three generators one entry at a time, so that any contiguous window of 250 entries contains 84/83/83 seeds of v2/GT/GP to within one seed, whatever its offset.

4.2. Evaluation convention of the pools

Each pool line stores a permutation together with its interval makespan. Both interval conventions of Section 3 occur in this literature, and the pools had been written under LEX2: job completion times aggregated lexicographically by upper bound. The solvers evaluate component-wise. The two agree on the upper bound — identical on 1,024/1,024 lines of every pool — and differ on the lower bound, which LEX2 leaves optimistic on 1–22% of lines per pool, by up to 98 time units on the 50×20 instances. We therefore re-evaluated every pool component-wise, leaving the permutations untouched, so that stored and solver-side objective values coincide throughout. The released pools carry the component-wise values.

4.3. Injection mechanism

Seeding is deliberately minimal. A seeded arm with fraction $k/250$ replaces the first k individuals of the initial population by pool lines; the remaining $250-k$ individuals are generated by the solver's standard random initialisation. For paired run $r \in \{0,\ldots,29\}$ the arm takes pool lines $[(r\cdot k) \bmod 1024,\ (r\cdot k+k) \bmod 1024)$ — consecutive blocks, no reordering or filtering, so each run draws from the generator's true output distribution rather than from a curated subset. The 30 blocks start at 30 distinct offsets and consecutive ones are disjoint, but with $k = 250$ and 1,024 pool lines the wraparound makes any two of them share 22% of their entries on average, and each pool line is drawn by 7.3 of the 30 runs; the seeded populations are therefore not fully independent across runs, which is one reason the unit of analysis throughout is the instance rather than the run. Scout-bee replacements during the run remain fully random. Seeds enter as raw permutations and are re-decoded and re-evaluated by the receiving solver, so no foreign objective values ever enter the search.

5. Pilot study: three ways to fool yourself

We first ran a full pilot on three clean instances (tai15_15_05, tai20_20_02, tai30_20_04; lower bounds 1224, 1581 and 1952) with six solvers (the portfolio plus two isolation variants), seven arms and 30 paired runs per cell. The pilot's role in this paper is methodological: each of its three phases exposed an artefact that the final design of Section 6 removes.

5.1. Artefact 1: stagnation-triggered stopping

Under the customary rule of 25 consecutive non-improving generations — the stopping criterion of every published solver in the portfolio — seeded GA arms terminated at the 25-generation floor (the solver could not improve the injected solutions), while the unseeded control ran for 100–150 generations. The resulting "final" comparison charges the seeds with the solver's inability to improve them. Any seeding study that uses stagnation-based stopping inherits this bias.

5.2. Artefact 2: budget-dependent gains that reverse

Re-running everything under a fixed budget of 200 generations (no stagnation rule) appeared to settle matters: seeded GA arms beat the control on the hardest instance by 34 units of expected makespan. A convergence check, however, showed the GA control still improving at budget exhaustion. At 5× the budget (1,000 generations) the control kept descending, overtook the stagnant seeded populations, and the effect reversed sign (from −34 to +67 on tai30_20_04 with full v2 seeding). The same check showed the elitist ABC's seeded advantage *persisting* at convergence (−46 to −35 with GP seeds), and the strong local-search solvers saturating at equal quality. A separate isolation experiment — attaching the identical tabu local search of TS-N2 to a GA frame — initially suggested the GA frame benefited from seeds where the ABC frame did not; extending its time cap dissolved that difference too, tracing it to budget starvation rather than to the frame. *Conclusion: no seeding claim about final quality is meaningful without a convergence check of the unseeded control.*

5.3. Artefact 3: what "faster" means

We measure convergence speed with two deliberately different metrics: (A) time to reach within 1% of the *arm's own* final value, and (B) time to reach the *unseeded control's* final quality (tolerance +0.5%). Metric A is self-referential: an arm that settles quickly on a worse plateau scores well, so an apparent acceleration may be nothing more than earlier stagnation. Metric B removes that degree of freedom by fixing a common target, at the cost of being undefined on instances where the arm never reaches it — which makes the proportion of instances reaching the target as much a part of the result as the time itself. Reporting both, with that proportion, prevents the most common over-claim in anytime comparisons.

5.4. What survives the artefacts

With all three corrections in place, the pilot's surviving picture is: (i) seeding improves converged quality for the elitist ABC on all three instances (Wilcoxon,

paired); (ii) for solvers with strong local search the final quality saturates but metric-A acceleration remains; (iii) for the plain GA the apparent gain is an artefact; (iv) GP and v2 seeds are statistically tied at the top of the generator ladder, MOR seeds never help; and (v) initial-population quality ranks exactly as the pool ladder predicts. These five statements were fixed before the final experiment and are what it tests at scale; Section 7.11 reports which of them survived.

6. Final experimental design

6.1. Instances

All 60 interval Taillard instances TA1–TA70 excluding TA11–TA20, plus ft10 (61 total). TA11–TA14 are the RL training set and TA15–TA20 the RL/GP development set; excluding them is mandatory because they contaminate the *seed generators*. Twenty instances used for irace tuning of TS-N2's hyper-parameters are retained but flagged: solver-tuning contamination applies equally to every arm of that solver and cancels in within-solver paired comparisons; a sensitivity analysis replicates all conclusions with the flagged instances removed. Size classes range from 15×15 to 50×20 (225–1,000 operations). Taillard's largest class, TA71–TA80 (100×20, 2,000 operations), is also excluded: at twice the operation count of our largest class it falls outside the budget calibration of Section 6, and extending the design to it would require a separate calibration that we did not perform. Consequently no seed pools were generated for either excluded group.

6.2. Arms

Seven arms per solver: the unseeded control A0; the generator ladder MOR, GT, GP, v2 and mix at the pre-specified fraction p^*; and a v2@50% contrast arm. The fraction sub-study (Phase 0, pre-specified decision rule combining converged quality in ABCE3 and time-to-target in TS-N2 over eight stratified instances) selected $p^* = 100\%$: the benefit of v2 seeding is monotone in the fraction on both criteria, with no interior optimum.

6.3. Budgets and stopping

Budgets are set per *(solver, size class)* rather than per instance. For each class we take one or two calibration instances, run the unseeded control on them, and record t_{conv}: the first CPU time at which the run-averaged trace reaches within 0.1% of its own final value. The budget applied to every cell of that class is

$$B(\mathrm{solver}, \mathrm{class}) = \max\bigl\{60\,\mathrm{s},\ \min\{1.5\,\tilde{t}_{\mathrm{conv}},\ 900\,\mathrm{s}\}\bigr\}, \tag{1}$$

where $\tilde{t}_{\mathrm{conv}}$ is the median over that class's calibration instances. The floor of 60 s prevents degenerate budgets on the small classes, where several solvers plateau in a few seconds. Every arm of a cell receives the identical budget, which is what the within-solver comparisons require. The eight calibration instances — `tai15_15_01`, `tai20_20_02`, `tai30_15_01`, `tai30_20_04`, `tai50_15_01`, `tai50_15_05`, `tai50_20_01` and `tai50_20_05` — are stratified across the size classes and are themselves members of the 61-instance evaluation set; the control traces behind t_{conv} comprise five runs for the GA and fEABC-LS and thirty for ABCE3 and TS-N2. The horizon of a class is therefore the control's own convergence timescale, measured on one or two of the instances to which it is later applied. What the endpoint measures is thus performance at a control-calibrated horizon rather than a solver-neutral approximation to convergence, a distinction we return to in Section 7.10.

The cap of 900 s is a feasibility constraint rather than a methodological choice. The design comprises 51,240 runs; at the budgets of (1) it consumed roughly 4,900 CPU-hours and fourteen days of wall-clock time on fourteen concurrent workers. Raising the cap would scale that figure proportionally on the classes that reach it, which are also the slowest. We therefore accept the cap and treat its consequence explicitly: on a substantial fraction of cells the unseeded control is still improving when the budget expires, so those cells cannot support a converged-quality claim. Section 7.2 identifies them and reports the final-quality results both over all instances and restricted to the cells that do satisfy the plateau rule.

6.4. Statistics

30 runs per cell. Run r of every arm of a solver is launched with the same RNG seed index, which blocks the design by (instance, run index). This is *not* a common-random-numbers coupling: a seeded arm does not draw from the generator to build its first k individuals, so the generator state diverges between arms immediately after initialisation and the runs follow unrelated random trajectories thereafter. The pairing is therefore a labelling of blocks rather than a variance-reduction device. Signed-rank tests on such pairs remain valid — under the null the per-instance differences are still symmetric about zero — but they are conservative rather than more powerful, and we make no claim of matched trajectories. Within-solver comparisons use paired Wilcoxon signed-rank tests with Holm correction per family, plus Vargha–Delaney $\hat{A}_{12}$ effect sizes [15, 16]; generator rankings use Friedman tests with post-hoc analysis. Anytime results are reported as expected-makespan-versus-CPU-time curves and as the two time-to-target metrics of Section 5, the latter accompanied by the proportion of instances on which the target is reached — which carries the information a success-rate curve would convey at the budget limit.

7. Results

All results below come from the full factorial design of Section 6: 61 instances $\times$ 7 arms $\times$ 4 solvers $\times$ 30 RNG-paired runs (51,240 runs). Every instance–arm cell contains exactly 30 runs; no cell was imputed or dropped.

7.1. Reference for relative deviations

Relative percentage deviation (RPD) is measured against published lower bounds rather than against best-known values produced by our own group, which would bias RPD downwards. Our interval instances are obtained from Taillard's benchmark [13] by a symmetric perturbation of each duration; we verified across all 57,450 operations that $\mathrm{mid}(p_{ij}) = c_{ij}$ exactly and that routings are unchanged. Because the makespan is composed only of additions and

maxima, and because $\mathrm{mid}(A+B) = \mathrm{mid}(A) + \mathrm{mid}(B)$ while $\mathrm{mid}(\max(A,B)) \geq \max(\mathrm{mid}(A), \mathrm{mid}(B))$ under the componentwise maximum, induction over the schedule DAG yields $\mathrm{mid}(C^{\mathrm{int}}_{\max}(x)) \geq C^{\mathrm{crisp}}_{\max}(x)$ for *every* schedule x, and therefore

$$\mathbb{E}[C_{\max}]^* \;\geq\; C^{\mathrm{crisp}*}_{\max} \;\geq\; \mathrm{LB}, \tag{2}$$

so the crisp lower bound is a valid (if not tight) bound for the interval problem. We take LB from the consolidated table of Coupvent des Graviers et al. [17]; the instance mapping was checked rather than assumed, with 19 of 60 instances showing exact agreement between the published bound and the machine-load bound recomputed from the instance files. For the remaining 41 the check is an inequality, which is necessary but does not identify an instance, and we do not claim more than that.

An exact identification is nevertheless available to any reader, and we supply what it needs. Because each interval was built as $[p_o - \delta,\, p_o + \delta]$, it is symmetric about the original crisp duration, so the midpoint recovers that duration exactly; across the 61 instances every midpoint is integral, with no exception. We therefore release the recovered crisp matrices, one per instance with a hash, so that identification reduces to an element-wise comparison against Taillard's published files. The recovery procedure itself is validated on the one instance of the set whose crisp form is unambiguous: `ft10` recovers to the Fisher–Thompson 10×10 matrix exactly.

Inequality (2) also yields a falsifiable prediction: no run may ever fall below its bound. Across all 51,240 runs there is not a single violation. This is a consistency check rather than a proof of correctness: many errors would preserve the inequality, and it is the released crisp matrices, not this check, that would settle the mapping. Moreover the bound is *attained exactly* on six instances — `tai30_15_05`, `tai50_15_01`, `tai50_15_02`, `tai50_15_04`, `tai50_15_07` and `tai50_20_09` — where our portfolio reaches $\mathbb{E}[C_{\max}]$ equal to the proven crisp optimum. By (2) these schedules are therefore optimal for the interval instance as well, which both certifies optimality on those six instances and confirms that

the inequality is tight rather than vacuous.

7.2. Which cells converged

The plateau rule of Section 6 — improvement in $\mathbb{E}[C_{\max}]$ below 0.1% over the final tenth of the budget — is applied to each control *run* rather than to the run-averaged curve, since averaging thirty traces can hide a subset that is still improving, and a cell is classified as converged when at least 90% of its runs satisfy it. The final value of a run is the solution the solver returns, not the last point of its trace. This classifies 111 of the 244 control cells as *anytime-only*: 49 of 61 for the GA, 25 for fEABCLS, 22 for TSN2 and 15 for ABCE3.

The distribution is informative in itself. The GA fails the rule on every instance of the four largest classes. The 50×20 class defeats every solver but ABCE3, which converges on five of its ten instances; ABCE3's own failures are otherwise confined to 15×15, where a third of its runs are still improving when the 60 s floor stops them. Classification is sensitive to both the threshold and the required fraction of runs: the count of converged cells moves from 114 at 0.05% to 133 at the 0.1% used here and 162 at 0.25%, and from 194 to 63 as the required fraction goes from half the runs to all of them. What matters is whether the *conclusions* move with it, and we report that directly below rather than claiming a stability the counts do not have. Per-cell classifications are released with the data.

Two properties of the rule need declaring. It is evaluated on the control only, so a cell that passes is one whose *control* has plateaued, not one in which every arm has; the comparisons that depend on convergence are therefore reported below on the intersection, where the control and the arm being compared both pass. And on TS-N2, 325 of the 1830 control runs record a *negative* improvement, down to -1.95%. The cause is a mismatch of objective rather than a defect: the framework's statistic takes the best individual under the solver's own ranking, which for TS-N2 is LEX2, so the recorded incumbent's midpoint need not be monotone even though its LEX2 value is. Those runs pass a rule that asks for improvement below 0.1% for a reason that has nothing to do with

Table 2: Mean RPD (%) over all 61 instances and over the subset whose *control* satisfies the plateau rule. Best arm per row in bold. The restricted rows are a common subset, so the arms remain comparable to each other, but a control-plateau instance is not one in which the seeded arm has necessarily plateaued as well; Table 3 gives the paired intersections, which is the stronger comparison.

Solver	Subset	A0	V2H	V2	MOR	GT	GP	MIX
GA	all (61)	**7.52**	7.66	8.05	9.70	8.47	7.99	7.58
	converged (12)	4.75	4.28	4.39	6.82	5.35	4.49	**4.00**
ABCE3	all (61)	10.51	9.51	9.39	10.71	10.20	9.68	**9.37**
	converged (46)	11.61	10.46	**10.34**	11.85	11.28	10.72	10.35
fEABCLS	all (61)	3.67	**3.66**	**3.66**	5.16	4.47	4.04	3.67
	converged (36)	3.39	**3.34**	**3.34**	4.80	4.07	3.73	3.40
TSN2	all (61)	3.60	3.64	3.62	4.25	3.99	3.67	**3.41**
	converged (39)	**2.75**	2.82	2.83	3.54	3.41	3.07	2.78

convergence, which inflates TS-N2's count of converged cells by an amount we cannot separate cleanly.

Table 2 shows that the restriction leaves two of the four solvers unchanged in every respect that matters. fEABCLS keeps the same best arm and the same ordering. ABCE3 keeps its conclusion — V2 and MIX are separated by a hundredth of a point and both beat the control by more than a point — while its absolute level rises, because the cells it fails are the small 15×15 ones on which it does best.

Because the rule is evaluated on the control alone, the sound comparison is the one restricted to instances where *both* the control and the arm being compared plateau. Convergence rates differ enough between arms to make this matter: on fEABCLS the mixed pool plateaus on 44 of 61 instances against the control's 36, while on the GA the randomised Giffler–Thompson pool plateaus on only 8. Table 3 reports those paired intersections.

The GA is the first exception, and the difference is not cosmetic. Over all

61 instances the unseeded control is the best arm (7.52 against MIX's 7.58); over the 10 instances where control and mixed pool both plateau, the ordering reverses and the mixed pool wins by 0.69 points (3.45 against 4.14). The two other pools that help at convergence, V2H and GP, reverse by a similar margin, while MOR and GT remain harmful there (+2.10 and +1.28). The negative result for the GA is therefore a statement about *insufficient budget*, not about seeding as such: it is the solver that needs the most time to consolidate a seeded population, and on the cells where it is given that time the harm disappears for the pools that were never the problem. This revises the reading of Section 8.2: what the GA lacks is not compatibility with heterogeneous parents in principle, but enough generations to exploit them under these budgets.

TSN2 is the second exception, and it cuts the other way. Over all 61 instances its mixed pool is the best arm (3.41 against the control's 3.60); over the 35 instances where both plateau, the margin vanishes entirely (2.34 against 2.32). The cells TSN2 fails are its largest — all ten 50×20 instances and eight of the ten 30×20 — which are precisely where Section 7.3 locates its gain. The benefit seeding brings to the strongest solver is thus confined to cells where its budget did not suffice to converge, a materially weaker claim than the unrestricted table supports, and we report it as such.

The conclusions are not equally robust to how the rule is set. Varying the threshold over $\{0.05, 0.1, 0.25\}\%$ and the required fraction of runs over $\{0.5, 0.8, 0.9, 1.0\}$, the best arm on the GA is the mixed pool in eleven of the twelve combinations, and on ABCE3 it is always a seeded arm. On TSN2 it is not stable: the mixed pool wins when half the runs must satisfy the rule, and the unseeded control wins when four fifths or nine tenths must. We therefore state the TSN2 result as conditional on that choice rather than as a finding.

Unless stated otherwise, the tables that follow use all 61 instances, so that every solver is evaluated on the same instance set; results restricted to converged cells are reported wherever they change a conclusion, which is the GA and TSN2.

7.3. Final quality at the budget

Table 4 reports mean RPD over the 61 instances. Three regimes are immediately visible, and they do *not* order themselves by solver strength.

Table 5 gives the corresponding paired tests. For each solver and arm we count the instances on which the arm differs significantly from A0 (paired Wilcoxon signed-rank, Holm-corrected within each instance across the six seeded arms, $\alpha = 0.05$), together with the mean signed difference in $\mathbb{E}[C_{\max}]$ and the aggregated Vargha–Delaney $\hat{A}_{12}$.

ABCE3: large and uniform gains.. The elitist bee colony is the weakest solver in the portfolio (10.51% RPD unseeded) and benefits massively. MIX and V2 each win on 35 of 61 instances and V2H on 32, and all three *lose on none*, with $\hat{A}_{12}$ between 0.266 and 0.298. Even GP, a mid-ranked generator elsewhere, wins 30 and loses none. This is the textbook case: the solver's own diversification never reaches the quality already present in the seeds.

TSN2: a gain confined to the largest instances.. The tabu-search hybrid is the strongest solver, and the natural expectation — which our own pilot supported — is saturation. It does not saturate, but the effect must be stated carefully. MIX wins 15 instances and loses 5 ($\Delta = -5.5$, $\hat{A}_{12} = 0.436$), lowering RPD from 3.60% to 3.41%, i.e. removing 5.3% of the excess over the lower bound. It is also the only arm that improves this solver: every other pool is neutral or harmful. In aggregate the gain is small, and it is *not* uniform: MIX is better on 35 instances and worse on 22, with a median relative change of -0.03%. The gain is almost entirely attributable to one size class, and we quantify this in Section 7.5. The defensible claim is therefore not that seeding improves the strongest solver in general, but that it does so on large instances under a search budget that is tight relative to instance size.

fEABCLS: neutrality.. The memetic ABC is statistically indistinguishable from its control under the three best generators (MIX 12/38/11, V2 7/44/10, V2H 7/43/11, all $\hat{A}_{12} \approx 0.50$). Its hill-climbing component reaches seed-level quality

unaided within the budget. Note that neutrality here is a property of the *good* pools only: MOR wins nothing and loses 56 of 61.

GA: harm, but only under an insufficient budget.. Over all 61 instances no arm improves the genetic algorithm: the best, MIX, is 13/29/19 with $\hat{A}_{12} = 0.527$, and V2 is a clear loss (9/25/27). The qualification of Section 7.2 is essential here. The GA fails the plateau rule on 49 of 61 instances — more than any other solver — and on the 12 where its control does converge the ordering reverses, with MIX at 4.00% against the control's 4.75%. The reversal in the pooled table is therefore a statement about the budget this solver needs, not about seeding being harmful to it in principle.

Across all four solvers the generator ranking is stable in its extremes: MOR is the worst seeded arm and GT the second worst on every solver (MOR reaches $\hat{A}_{12} = 0.920$ on the GA and 0.885 on fEABCLS), while MIX holds the best Friedman rank among seeded arms on all four (Section 7.9). On mean RPD, MIX is the best seeded arm on three solvers and is within 0.01 points of the best on fEABCLS. The positions in between are not stable: GP outranks V2 on the GA but not on the other three. The learned policy is thus competitive with the best tuned heuristic without dominating it, and it is the mixed pool rather than any single generator that is uniformly at the top — which already suggests that *pool diversity* rather than raw seed quality drives the benefit. Section 7.8 tests that directly.

7.4. Effect sizes with the instance as unit

Table 5 counts, per instance, whether an arm differs from the control. That summary is easy to read but it does not control error across the study: it runs six tests per instance and solver, corrects only within each instance, and then tallies the outcomes. We therefore repeat the analysis with the instance as the experimental unit throughout. Each arm is summarised per instance by the mean of its 30 runs; the quantity analysed is the per-instance difference from the control, giving $n = 61$ observations; one signed-rank test per (solver, arm)

is applied to those differences, with Holm correction over the six arms of each solver; and a 95% confidence interval for the mean difference is obtained by bootstrapping instances (10^4 resamples). Table 6 reports the result.

Three readings change relative to the per-instance tally.

ABCE3 is unaffected by the stricter analysis.. MIX improves the solver by 1.14 percentage points of RPD with a confidence interval of $[-1.36, -0.93]$ and $p < 10^{-3}$ after correction; V2, V2H and GP are likewise unambiguous. Correcting over all 24 contrasts of the study rather than within solvers changes nothing: the same 15 remain significant.

The GA is not harmed by the mixed pool.. Its difference is $+0.055$ points with an interval spanning zero and $p = 0.55$. What is significantly harmful on this solver is every *single-generator* pool: MOR (+2.18), GT (+0.95), V2 (+0.53) and GP (+0.47). Combined with Section 7.2, the picture is that the GA is damaged by homogeneous seeds under an insufficient budget, not by seeding as such.

The improvement on TSN2 does not reach significance.. This is the reading that changes most. The signed-rank test over the 61 per-instance differences gives $p = 0.065$ before correction and $p = 0.26$ after, so MIX cannot be declared better than the control by a test that weights every instance equally. The bootstrap interval for the mean difference, $[-0.35, -0.045]$, nonetheless excludes zero. The two statistics are not in conflict: the signed-rank test asks about the median difference, which is essentially zero because 51 of the 61 instances are unaffected, while the mean is pulled negative by the ten 50×20 instances that carry 94.8% of the effect (Table 7). We therefore state the TSN2 result as what it is — a small mean improvement concentrated in one size class, with an interval that excludes zero but without significance under an instance-weighted test — and not as a general improvement of the strongest solver. Establishing the latter would require either more instances in the affected regime or a budget under which more of them behave like 50×20.

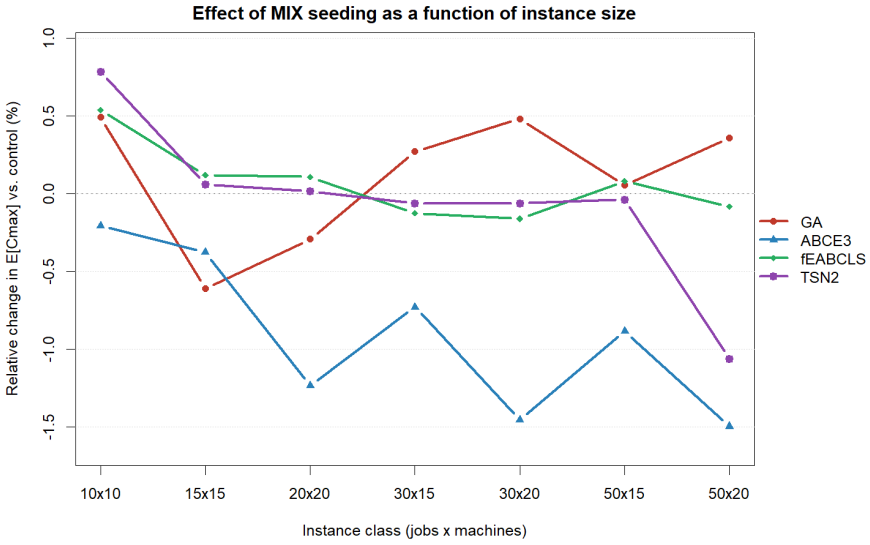

Figure 1: Relative change in $\mathbb{E}[C_{\max}]$ under MIX seeding with respect to the unseeded control, by instance class (ordered by number of operations). Negative values favour seeding. The benefit for ABCE3 and TSN2 grows with instance size, whereas the penalty for the GA persists.

7.5. The effect scales with instance size

Aggregate figures conceal a strong interaction with problem size. Figure 1 plots the relative change in $\mathbb{E}[C_{\max}]$ under MIX seeding against instance class, ordered by number of operations (100 to 1000).

Two trends emerge. For ABCE3 the gain deepens with size, from -0.21% on the smallest instance to -1.24% on 20×20 and -1.50% on 50×20. For TSN2 the effect is flat and near zero up to 50×15 and then drops sharply to -1.06% on 50×20 — and on that class *every* seeded arm beats the control, including GP ($\Delta = -30.9$), MOR ($\Delta = -25.6$) and GT ($\Delta = -19.6$), all three of which are harmful everywhere else. The GA moves in the opposite direction, remaining between $+0.05\%$ and $+0.48\%$ on the four largest classes.

Table 7 makes the concentration explicit for TSN2, and it is the single most important caveat in this paper. The ten 50×20 instances account for 94.8% of the aggregate improvement; the remaining 51 instances contribute almost noth-

ing. Reporting only the pooled figure of Table 4 would therefore misrepresent the result as a general improvement of the strongest solver, which it is not.

The interpretation is a budget-starvation argument. On 50×20 instances (1000 operations) even a strong tabu search cannot, within the 900 s cap, recover from a random start the structure that a constructive heuristic supplies for free. Seeding does not make the search better; it removes a phase of the search that the budget cannot afford. This predicts that the benefit of seeding should be reported as a function of the budget-to-size ratio rather than as a single aggregate figure, and that published seeding gains obtained on small instances will not transfer.

7.6. Anytime behaviour

Final quality alone cannot distinguish a solver that is genuinely improved by seeding from one that is merely given a head start. Because per-solver budgets range from 60 s to 900 s across size classes, absolute time is not comparable across instances; we therefore normalise each trace to the fraction of its own budget consumed, the standard convention for aggregated anytime curves, and average over instances on a common grid so that no instance enters or leaves mid-curve. Figure 2 shows the resulting profiles over all 61 instances, and Table 8 gives the values at five budget fractions.

One caution applies to these curves. They record the best individual *in the population* at each sampling point rather than the solution the solver finally returns, and the last sample precedes termination by up to one sampling interval. In practice the two agree closely: the curve endpoint differs from the final quality of Table 4 by a median of 0.000% on ABCE3, 0.003% on fEABCLS and 0.029% on the GA. On TSN2 the curve ends 0.041% *below* the returned quality, because TS-N2 ranks internally by LEX2 while the curve tracks $\mathbb{E}[C_{\max}]$, so a schedule that is better on the midpoint need not be the one it returns. We nonetheless confine every cross-arm comparison to a single solver, where all arms are measured identically.

Within that scope, the aggregate view exposes a result the final-quality

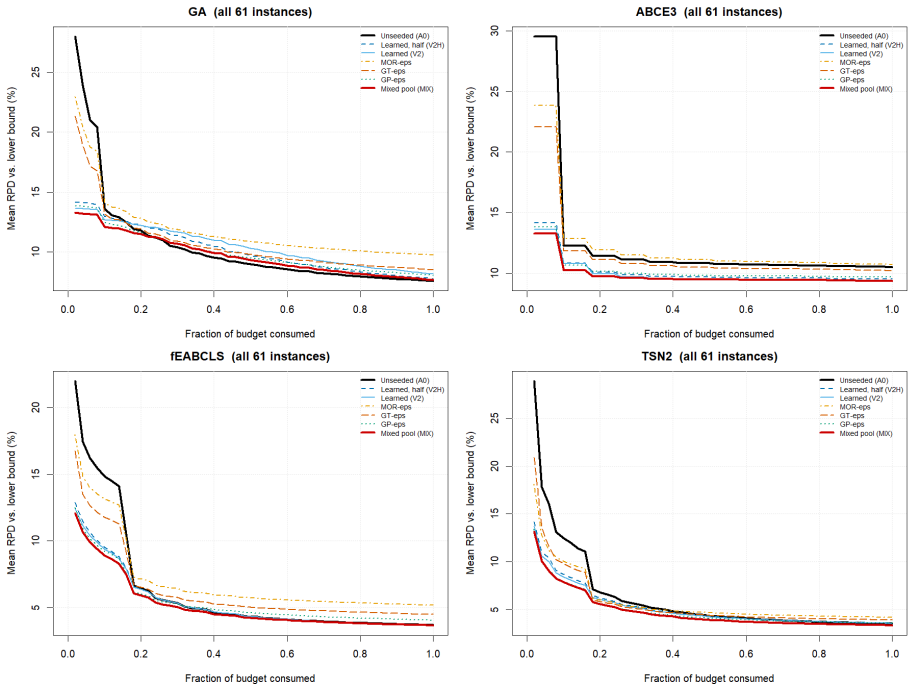

Figure 2: Anytime behaviour over all 61 instances. Mean RPD with respect to the published lower bound against the fraction of the per-instance budget consumed, for the unseeded control and the two best seeded arms.

tables cannot show: *at short budgets seeding is a large win for every solver in the portfolio, but by margins that differ by an order of magnitude between them.* At 10% of budget MIX cuts mean RPD from 14.84% to 8.91% on fEABCLS and from 12.48% to 7.89% on TSN2, but only from 12.28% to 10.23% on ABCE3 and from 13.58% to 12.10% on the GA — reductions of 40%, 37%, 17% and 11% in the excess over the lower bound, equivalent to 5.2%, 4.1%, 1.8% and 1.3% of makespan. The two largest short-budget gains therefore accrue to the two *strongest* solvers, which is the reverse of the ordering at the budget limit. Both are the solvers whose control spends its opening on an unseeded random population before local search can act: fEABCLS falls from 14.84% to 5.60% between a tenth and a quarter of its budget, and seeding buys precisely that opening. The practical reading is that the question "does seeding help?" is

ill-posed unless the budget is specified, and that which solver benefits most depends on where in its own trajectory the budget cuts.

The GA panel isolates the reversal. Its seeded arms lead until roughly 20% of the budget, after which the control crosses and stays below to the end. This is the anytime signature of the consolidation failure analysed in Section 8.2: the seeded population is ahead while quality is inherited from the seeds, and falls behind as soon as progress must come from recombination. ABCE3 shows the opposite profile, a gap that opens immediately and never closes.

The generator ranking is also stable along the whole trajectory, not merely at the end. MOR is the uppermost seeded curve in all four panels and GT the next, so their disadvantage is not an artefact of where measurement stops: from a quarter of the budget onwards both are worse than the control on the two ABC variants. At a tenth of the budget they are still ahead of it, which says more about how poor an unseeded random population is at that point than about any merit of theirs. MIX is the lowest curve in three of four panels for essentially the entire run. Two observations qualify the ranking. First, GP is competitive with MIX at very short budgets — on TSN2 at 10% of budget GP reaches 7.94% against MIX's 7.89% — so the advantage of the mixed pool *widens as the search proceeds* rather than being inherited from higher initial seed quality. That is exactly what one expects if the operative variable is pool diversity rather than seed quality, and Section 7.8 confirms it directly. Second, V2 degrades relative to V2H on the GA as the budget grows (8.14% versus 7.73% at convergence), a hint that seeding the entire population with a single learned policy is worse than seeding half of it, and that some random material is worth retaining.

Figure 3 restricts the same analysis to the 50×20 class in absolute time, where the seeding effect on the strong solvers is concentrated.

Two features are specific to this class. ABCE3's control flattens at approximately 11.2% while both seeded arms flatten near 9.7%, a gap present from the first seconds that never closes — a genuine improvement in converged quality rather than an acceleration. TSN2's control exhibits a sharp step near 50 s, when the tabu phase first engages, and thereafter descends steadily; MIX re-

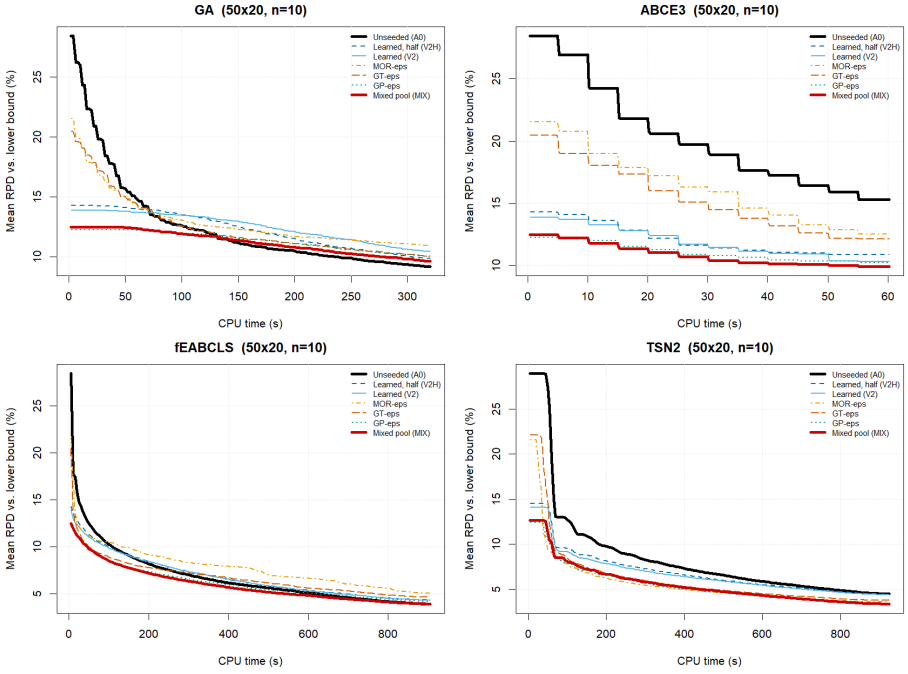

Figure 3: Convergence on the 50×20 class (10 instances, 30 runs). Mean RPD with respect to the published lower bound against CPU time, for the unseeded control and the two best seeded arms. Note the differing time axes: solver budgets are calibrated per solver and size class.

mains below it over the entire trajectory and is still below at the budget limit (approximately 3.3% against 4.1%). The separation is therefore not an artefact of where the budget stops.

One methodological consequence deserves emphasis. The GA result would have been reported as a *success* under any budget shorter than its crossover point, which is precisely the artefact documented in Section 5. Since seeding studies commonly fix a budget without verifying convergence, this is a plausible source of optimistic seeding claims in the literature, and it argues for reporting anytime profiles rather than single-budget summaries.

7.7. Time to target

Anytime curves show *that* seeded arms lead early; time-to-target quantifies *by how much*. We report the two metrics of Section 5: (A) the time to come within 1% of the arm's own final value, and (B) the time to reach the unseeded control's final quality within +0.5%. Both are expressed as a fraction of the per-instance budget so that classes with different budgets can be pooled, and both are medians over the 61 instances. Metric B is reported together with the proportion of instances on which the arm reaches the target at all, since a median computed only over successes is meaningless without it.

Metric A behaves exactly as the pilot warned. Its medians span 0.09 to 0.70 of the budget, so it is not that the metric fails to separate the arms; it is that it separates them in the wrong order. On the GA the earliest plateau belongs to MOR, at 0.49 against the control's 0.60, and on TSN2 the same, 0.27 against 0.36 — while the arms that actually finish best plateau latest, V2 at 0.70 on the GA. This is the signature of a self-referential target: an arm that settles quickly on a poor value scores well *because* it is poor. MOR is the worst generator on all four solvers, and metric A makes it look the fastest on two of them. We report it to document that it must not be used alone, and do not tabulate it further.

Metric B needs care of a different kind. An arm that never reaches the target on an instance has no time-to-target there, and dropping such instances would summarise only the successes — precisely discarding the cases where the arm does worst. The reach rate and the times must therefore enter the analysis together.

Two properties of the design constrain how. First, the arms are paired: each seeded arm and its control share the instance, the budget and the target, so a comparison that treats them as independent groups is not available. Second, and more restrictive, the target on an instance is the control's own endpoint, so the control attains it on all 61 instances by construction. Its sample cannot be censored, whereas the seeded samples can. A log-rank test against such a reference contrasts a censored sample with one that is uncensorable by definition, and we therefore do not report one.

What survives both constraints is a paired analysis of the recorded times, with non-reachers recorded at the budget, $T = 1.0$. That recording understates a non-reacher's true time, and since only seeded arms are ever non-reachers, the understatement runs in favour of seeding. It biases the mean but not the sign: an arm recorded at 1.0 has $T \geq T_{A0}$ and is always counted as later. We accordingly take the sign test on the paired earlier/later counts as the primary contrast, and report the paired difference in restricted mean time (ΔRMTT, with a bootstrap interval over instances) as a secondary quantity whose bias is known and runs in one direction.

Under this treatment ABCE3 stands out. Its mixed pool reaches the control's *final* quality on every instance and earlier on 57 of the 59 that are not ties, cutting restricted mean time from 0.38 to 0.12 of the budget; V2 and V2H do nearly as well. These are unambiguous "same quality, sooner" results, and they agree with the converged-quality finding for that solver. TSN2 joins it more modestly: its mixed pool is earlier on 42 instances and later on 15 ($p = 0.003$), and reaches the target on 93% of them against the control's 100% by construction.

The other two solvers give the opposite answer. *Five of the GA's six seeded arms are significantly slower* to the control's own final quality; only the mixed pool escapes, and narrowly (18 earlier, 37 later, $p = 0.057$), though its restricted mean time is clearly worse ($+0.09$, $[+0.04, +0.13]$). That is the same pattern the final-quality analysis found on this solver: every single-generator pool harms it and the mixed pool does not. On fEABCLS the three competitive pools — V2H, V2 and MIX — are statistically indistinguishable from the control ($p = 1.00$ for V2 and MIX), while the three weak ones are significantly slower. Seeding buys that solver nothing in time.

Mean and sign agree almost everywhere. They part on three arms: MIX on the GA, V2H on fEABCLS and GT on TSN2, where ΔRMTT reaches significance and the sign test does not. In each the difference is concentrated in a minority of instances rather than being a general shift, which is exactly what the sign test is there to detect.

Ten arm–solver pairs are significantly *slower* to the target than their control, and MOR is the extreme of every one: it is slower on all four solvers and reaches the target on no instance at all on the GA and fEABCLS, where 59 of 61 comparisons go against it. GT is slower on two solvers and, revealingly, *faster* on ABCE3 — the solver that tolerates every pool. Both generators sit at the bottom of the quality ladder of Section 7.8, so this is the one place in the study where seed quality and outcome move together.

The contrast with a success-only tally is instructive. Computing medians over successes alone would have credited MOR on the GA with reaching the target at 71% of budget, a figure derived from the 15% of instances where it succeeds at all.

7.8. Initial-population quality: the mixed pool does not start better

The obvious explanation for MIX's advantage would be that its seeds are simply better. They are not. Table 10 reports the quality of the initial population each arm actually receives, read from the step-zero record of the traces, so it is the solvers' own evaluation rather than the value stored with the pool.

On the mean of the initial population — the natural measure of pool quality — MIX is *worse* than both V2 and GP, by 1.6 and 2.4 percentage points respectively, and it never holds the best individual by a meaningful margin. It nonetheless finishes ahead of both (Table 4). A pool that starts worse and ends better is direct evidence that the operative variable is composition, not seed quality.

This also disposes of a possible confound. The mixed pool is assembled from exactly the same 1,024 solutions per generator that feed the V2, GT and GP arms — it introduces no new material, only a different mixture — so no difference in provenance or generation effort can be invoked to explain its advantage.

7.9. Which seeding strategy? Generator ranking and its solver-dependence

The practitioner's question is not whether seeding helps in general but which pool to use, and whether that choice must be re-made for every solver. We

answer both with a Friedman test over the seven arms with the 61 instances as blocks, followed by Holm-corrected paired post-hoc comparisons against the top-ranked arm of each solver. Table 11 reports the mean ranks.

Two findings follow, and they answer different questions.

Which generator: the answer varies little across solvers.. The ranking of the six seeded arms is highly consistent across the portfolio: restricted to those arms, the mean pairwise Spearman correlation between the four rank vectors is $\rho = 0.88$. MIX is the best *seeded* arm for all four solvers, with ranks between 2.41 and 2.95, and is never significantly worse than the top-ranked arm of any solver — indeed it is the top-ranked arm outright for three of the four. MOR is last for all four (5.37 to 6.52) and GT second-to-last for three. With six arms and four solvers a rank correlation of 0.88 is a descriptive summary rather than a demonstration of invariance, and the intermediate positions do move: GP outranks V2 on the GA but not elsewhere. What is stable is the top and the bottom, which is what a practitioner needs: within this portfolio, use a mixed pool and avoid the low-diversity constructive pools, without re-tuning that choice per algorithm.

Whether to seed at all: the answer does depend on the solver.. The only arm whose rank moves substantially across solvers is the unseeded control, from 2.69 on the GA (where it is the top-ranked arm overall) to 5.60 on ABCE3 (where it ranks sixth of seven, beaten by every seeded arm except MOR). Including A0 drags the mean pairwise agreement down from $\rho = 0.88$ to $\rho = 0.69$, and the weakest pair, GA versus ABCE3, rises from $\rho = 0.36$ to $\rho = 0.83$ once the control is removed. The displacement of A0 is thus the entire source of cross-solver disagreement.

This dissociation is the cleanest statement of the paper's result. The question "which seeds are good?" has a solver-independent answer, because it is a property of the pool. The question "should I seed?" does not, because it depends on whether the solver's variation operators can exploit the pool — the mechanism analysed in Section 8.2. Reporting only the first, as seeding studies

commonly do, produces a recommendation that silently fails on solvers like the GA.

Why the advantage is not an artefact of sampling.. A natural objection is that MIX might win because it exposes the search to more distinct seed material. Three properties of the design rule this out. The five pools are the same size ($L = 1024$), so every arm draws 30 blocks of 250 under the same arithmetic. The mixed pool is assembled from precisely the solutions that feed the V2, GT and GP arms, so it contains no material those arms do not also see. And directly measured (Section 7.8), the population MIX actually receives is on average *worse* than the ones V2 and GP receive, yet it converges to better solutions. Sampling breadth and seed quality both point the wrong way for the objection; within-population generator diversity is what remains.

7.10. A limitation of CPU-time budgets

The stopping rule is a fixed CPU-time budget, measured with the process clock. CPU-seconds are reproducible; the *work* they buy is not, because the instructions retired per CPU-second depend on contention for cache and memory bandwidth among the concurrently executing workers. All cells here were produced under an identical occupancy — fourteen concurrent workers throughout — so the effect is constant across the design, and within a solver it is in any case common to all seven arms, leaving the paired instance-level comparisons that carry every claim in this paper unaffected. What is *not* transferable is the absolute level: an RPD of 3.41% for TSN2 is specific to this machine at this occupancy and should not be compared against figures published for these solvers elsewhere. More generally, studies reporting a single absolute quality figure under a wall-clock or CPU-time budget, without stating the hardware occupancy under which it was obtained, are less reproducible than they appear.

7.11. The pilot's five statements, at scale

Section 5 closed with five statements fixed before the final experiment. Reporting their fate is part of the point of specifying them.

(i) Seeding improves converged quality for the elitist ABC — **confirmed**, and by a wider margin than the pilot suggested: 35 wins, 0 losses out of 61, with no seeded arm except MOR failing to help.

(ii) For solvers with strong local search the final quality saturates but metric-A acceleration remains — **saturation confirmed, acceleration refuted**. Saturation holds: fEABCLS is statistically unchanged at the budget. The acceleration does not survive either time-to-target metric. Metric A orders the arms by how early they settle rather than by where they settle, and under metric B the mixed pool on fEABCLS is statistically indistinguishable from its control — earlier on 28 instances, later on 26, $p = 1.00$ (Section 7.7). What remains is the budget-fraction table, where fEABCLS improves from 14.84% to 8.91% at a tenth of the budget. The gain at short budgets is therefore real, but it is not the general acceleration the pilot predicted.

(iii) For the plain GA the apparent gain is an artefact — **confirmed at the pilot's budgets, but incomplete**. The gain does reverse under a converged budget, as the pilot found. What the pilot could not see, having used three instances, is that on the 11 instances where the GA's control genuinely converges the reversal reverses again (Section 7.2).

(iv) GP and v2 tie at the top of the ladder and MOR never helps — **partly confirmed**. MOR never helps: it is last on all four solvers. But GP and v2 do not tie at the top; the mixed pool outranks both on every solver, which the pilot's design could not have detected because it is a property of combining them.

(v) Initial-population quality ranks as the pool ladder predicts — **refuted for the mixed pool**. The single-generator arms rank as expected, but MIX starts from a worse average population than V2 or GP and still converges better (Section 7.8). This is the one statement the final experiment contradicts outright, and it is the observation that reoriented the mechanism analysis.

7.12. Threats addressed by design

Before the checks themselves, one inferential property of the time-to-target design needs establishing rather than assuming. Its target is *endogenous*: it is drawn from the control's own endpoint, so the control is measured against a quantity taken from its own realisation while the seeded arm is not. That asymmetry is not obviously harmless, and it is not enough to observe that censoring cannot reverse a sign. We therefore measured it. Splitting each instance's thirty control runs at random into two halves of fifteen, averaging each half and treating one as control and the other as arm, reproduces the analysis under a null that is true by construction. Over 400 such replicates the sign test rejects at 4.3% on the GA, 6.8% on ABCE3, 4.0% on fEABCLS and 3.3% on TSN2 against a nominal 5%, so the procedure is close to calibrated. It is not exactly so: under the null the seeded arm counts as earlier on only 46.2%, 45.3%, 47.3% and 48.7% of instances respectively, a small tilt against seeding. Every sign test in Table 9 is taken against that solver's own null proportion rather than against 0.5, which is what removes the tilt; the effect is visible on the GA, where the mixed pool reads as significantly slower against 0.5 and does not against 0.462.

Three further checks guard the numbers above. First, expected makespans were recomputed by an *independent* verifier: a standalone program that shares no code with the solver. It re-implements the instance parser, the interval arithmetic and the decoding from scratch, depends only on the standard library, and includes no header of the framework, so a defect in the solver's scheduling or interval classes cannot hide behind it. The sample is systematic rather than random — every combination of the four solvers and seven arms on one instance from each of the seven size classes, 980 stored solutions in total — so each solver, arm and class is represented.

The verifier reproduces the reported interval makespan exactly in 979 of the 980 cases. This is the expected outcome once one accounts for Lamarckian replacement: after local search the framework re-encodes the improved schedule into the chromosome, so the stored sequence already carries the machine orders of the schedule that was actually executed, and decoding it returns that same

schedule. The single exception, one run on `tai50_20_09`, differs by 0.5 units of $\mathbb{E}[C_{\max}]$ on a value of 3119 — 0.016%, and in the direction that makes the reported figure the more pessimistic. We have not traced its cause and record it as an open point. Second, every arm of a solver runs on the same instance with the same seed indices and the same budget, so the design is blocked by instance and no arm is systematically advantaged; the qualification of Section 6 on what that pairing does and does not guarantee applies. Third, the stopping rule is a fixed CPU-time budget per size class, identical across arms, which eliminates the stagnation-triggered artefact documented in Section 5.

8. Discussion

8.1. Seeding quality is not the operative variable

The pre-specified hypothesis — that seeding helps precisely those solvers whose own search saturates below the seed level — is not supported. Ordering the portfolio by unseeded RPD gives ABCE3 (10.51) > GA (7.52) > fEABCLS (3.67) > TSN2 (3.60), but the seeding effect follows the order ABCE3 (large gain) > TSN2 (small gain) > fEABCLS (null) > GA (loss at the budget, gain once the budget suffices). The strongest solver in the portfolio benefits while a mid-ranked one appears harmed, so solver quality alone does not predict the sign of the effect.

Two variables do carry information, and the design supports them as associations rather than isolating them as causes. The first is the *budget relative to what the solver needs*. Section 7.2 shows the GA reversal to be a budget effect, and Section 7.5 shows the TSN2 gain confined to the class where the budget is tightest relative to instance size. The two findings point the same way: seeding matters most where the solver has least time to build structure itself, and its apparent sign can flip when that time is short enough.

The second is pool composition. MIX — equal thirds of v2, GT and GP, i.e. 84/83/83 of the 250 seeded individuals — holds the best Friedman rank among seeded arms on all four solvers, while the single-generator pools of comparable

individual quality (V2, GP) are weaker and the low-diversity pools (MOR, GT) are actively harmful. Since MIX is assembled from exactly the solutions that feed V2, GT and GP, and starts from a worse average population than either V2 or GP (Section 7.8), neither provenance nor seed quality can account for its advantage. What we cannot do is separate diversity from the other properties a mixture has — coverage of distinct basins, complementarity between specific pairs of generators — because we did not run pools matched on quality with diversity varied independently. The composition claim is therefore an association supported by an elimination of the obvious alternatives, not a controlled result.

8.2. Why the genetic algorithm is harmed: a failure to consolidate

The intuitive explanation for a seeding-induced loss is premature convergence: a population of good, similar individuals collapses onto a local optimum. The data do not support it. We instrumented the GA and ABCE3 with the mean pairwise Hamming distance of the population, sampled every 10 s over 10 runs on four medium instances (normalised to $[0, 1]$, where 0 denotes an identical population). Table 12 reports the result. The seeded GA is not less diverse than its control — it is *significantly more* diverse, by +24.3% under V2 and +13.7% under MIX (paired Wilcoxon, $p < 10^{-4}$), and it ends the run at 0.720 against the control's 0.464.

The failure is therefore the opposite of premature convergence: within a fixed budget the seeded GA never consolidates. Its control contracts by 51.0% in structural spread and exploits the region it has found; under V2 the seeded variant contracts by only 22.7% and is still exploring when the budget expires. The proximate cause is that the seeds, being drawn from generators with different construction logic, are good but mutually *dissimilar*; JOX recombining two structurally divergent parents produces offspring that inherit the operation ordering of neither, so selection cannot accumulate structure. The harm grows with instance size (Section 7.5) because longer chromosomes need more generations to consolidate, and the budget does not grow proportionally.

The contrast with ABCE3 is consistent with this reading. Seeding does not

perturb ABCE3's diversity trajectory at all (differences of -2.3% and $+4.0\%$, $p = 0.27$ and $p = 0.90$), and its population stays dispersed throughout (0.85 at the end). ABCE3 does not rely on recombining population members to build structure, so seeds are retained and refined rather than destroyed.

The ordering of the two seeded arms is itself informative. V2, drawn from a single generator, produces the least consolidation (-22.7%) and the largest loss in final quality; MIX, the more heterogeneous pool, consolidates further (-40.3%) and is the better of the two arms on this solver. Structural heterogeneity of the *pool* and of the *surviving population* are therefore not the same quantity: what slows the GA is a population its operator cannot consolidate, not a diverse pool as such.

Three limits on this account should be stated plainly. First, it is consistent with the convergence analysis rather than independent of it: Section 7.2 shows the GA's loss disappears where its control converges, which is what a consolidation-rate explanation predicts, but both observations rest on the same runs. Second, GA and ABCE3 differ in selection, replacement, mutation and population dynamics as well as in recombination, so attributing the contrast to JOX specifically is under-determined; establishing it would require holding the rest of the GA fixed and varying the crossover, which we did not do. Third, mean pairwise Hamming distance is a genotypic measure on a representation with repeated symbols, and need not track the building blocks JOX actually manipulates; a measure of parent–offspring precedence preservation would test the mechanism more directly. We therefore offer consolidation failure as the reading the present data favour, not as a demonstrated cause.

8.3. Practitioner guidance

The results support three recommendations, each with its scope attached. (i) Seed with a *mixed* pool: it outranks every single-generator pool on all four solvers. We did not measure generation cost, so we make no claim that combining three generators is free; a practitioner working at very short budgets, where seeding matters most, should measure it. (ii) Expect the benefit to grow as the

budget tightens relative to instance size, and expect its *sign* to be unreliable when the budget is far from sufficient — that is where our genetic algorithm reverses. (iii) Before concluding that seeding fails on a recombination-based solver, check whether its control has converged; on our data that distinction separates a genuine null from an artefact of the budget.

8.4. Limitations

Scope.. The study covers one problem family (interval JSP), one encoding (permutation with repetition), and four solvers three of which share the ABC framework or components of it. Nothing here licenses a general statement about recombinative metaheuristics or about seeding at large; the conclusions are about this problem, this representation and these operators. Replication on the deterministic JSP, on a different uncertainty model, or on an external benchmark would be the natural next test.

Mechanism.. The two explanatory claims — pool composition rather than seed quality, and consolidation failure rather than premature convergence — are associations that survive the obvious alternatives, not controlled results. Establishing them would require pools matched on quality with diversity varied independently, and a GA in which only the crossover changes. Neither experiment is in this design.

Budgets and cost.. Budgets are CPU-time caps measured with the process clock on a single Linux machine at a fixed occupancy of fourteen concurrent workers. Within a cell every arm shares the cap, but absolute times are not portable, and the work a CPU-second buys depends on that occupancy. We also do not measure the cost of *producing* the pools: the training of the learned policy is excluded as amortised across instances, and the per-instance generation cost of the four generators was not instrumented. At the budgets where seeding matters most — a tenth of ours — that omission is material, and we flag it rather than assume it away.

Design gaps.. The seeded fraction $p^* = 100\%$ was selected on two solvers and eight instances, so it is a protocol decision for this study rather than a generally preferable fraction; the GA's V2H arm, which is less harmful than V2, suggests the fraction interacts with the solver. The mixed pool was not compared against two-generator mixtures or against alternative proportions, so we cannot attribute its advantage to any particular pairing. And the structural-diversity instrumentation covers four instances and two solvers, a narrow base for the argument it supports.

9. Conclusions

We conducted, to our knowledge, the largest controlled study of population seeding for the interval job shop: four solvers, seven initialisation arms, 61 benchmark instances and 30 runs per cell — 51,240 runs — evaluated against published lower bounds under a fixed-budget protocol designed to eliminate three artefacts we document in a pilot study.

Seeding is not uniformly beneficial, and its sign is not predicted by solver strength. Analysed with the instance as the unit, a mixed pool improves the weakest solver decisively (-1.14 points of RPD, 95% CI $[-1.36, -0.93]$), leaves a memetic ABC unchanged, and does not harm a genetic algorithm, though every single-generator pool does. On the strongest solver the mean improvement is small, concentrated in one size class, short of significance under an instance-weighted test, and absent altogether on the instances where that solver's own control converges; we report it as a bounded result rather than as an improvement of the state of the art.

Three findings seem to us to travel beyond this problem family. First, *the budget decides the sign, not just the size.* The two effects at the ends of our portfolio both move when the analysis is restricted to cells whose control actually converged, and they move in opposite directions: the genetic algorithm's apparent damage disappears, and the tabu hybrid's gain disappears with it. Its benefit is confined to the largest instances, which are exactly the cells where

its budget did not suffice to converge. Neither reversal is visible in a table that pools converged and budget-limited cells. A seeding study that fixes one budget without checking convergence can therefore report either sign, and which one it reports is not predictable from the solver's strength.

Second, *composition beats quality.* The mixed pool is assembled from exactly the solutions that feed the single-generator arms and receives a worse average initial population than two of them, yet it holds the best Friedman rank among seeded arms on all four solvers. Seed quality is not irrelevant at the bottom of the ladder: MOR, the weakest generator, is significantly slower to the control's own final quality on all four solvers, and GT on three of them. But among the pools that help, quality does not order them, and composition does.

Third, *the failure mode is not the one usually assumed.* Premature convergence is the standard explanation for seeding going wrong; our diversity measurements show the opposite, a population that stays dispersed and never consolidates within the budget. We advance consolidation failure as the reading the data favour, while noting that separating it from the other differences between the two solvers would need ablations this design does not contain.

The practical consequence is that seeding should be treated as an interaction between pool composition, variation operator and budget, evaluated at the instance size of the intended application, rather than as a property of the seeds alone. And the methodological consequence is that the three habits our pilot identified — stagnation-triggered stopping, single-budget comparison, and self-referential speed metrics — are enough to manufacture a positive seeding result where none exists, or to hide one that is there.

Data and code availability

The seed pools, the injection operator, the experiment setups, the run logs of all 51,240 runs, the aggregated result tables and the analysis scripts that produce every table and figure in this paper, together with the recovered crisp instance matrices of Section efsec:rpd-ref and their hashes, are deposited at

[DOI pending], together with the independent verifier of Section 7.12 and the commit hash of the solver framework.

References

[1] D. Lei, Population-based neighborhood search for job shop scheduling with interval processing time, Computers & Industrial Engineering 61 (2011) 1200–1208. doi:10.1016/j.cie.2011.07.010.

[2] D. Lei, Interval job shop scheduling problems, International Journal of Advanced Manufacturing Technology 60 (2012) 291–301. doi:10.1007/s00170-011-3600-3.

[3] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU 2020), Vol. 1238 of CCIS, Springer, 2020, pp. 663–676. doi:10.1007/978-3-030-50143-3_52.

[4] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Elite artificial bee colony for makespan optimisation in job shop with interval uncertainty, in: Bio-inspired Systems and Applications: from Robotics to Ambient Intelligence (IWINAC 2022), Vol. 13259 of LNCS, Springer, 2022, pp. 98–108. doi:10.1007/978-3-031-06527-9_10.

[5] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, Integrated Computer-Aided Engineering (2023). doi:10.3233/ICA-230705.

[6] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, Natural Computing 22 (2023) 645–657. doi:10.1007/s11047-023-09953-2.

[7] H. Díaz, Neighbourhood structures and ranking operators for the interval job shop scheduling problem, under review, Computers & Operations Research (2026).

[8] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, X. Chi, Learning to dispatch for job shop scheduling via deep reinforcement learning, in: Advances in Neural Information Processing Systems, Vol. 33, 2020.

[9] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: a methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421.

[10] B. Kazimipour, X. Li, A. K. Qin, A review of population initialization techniques for evolutionary algorithms, in: IEEE Congress on Evolutionary Computation (CEC), 2014, pp. 2585–2592.

[11] S. J. Louis, J. McDonnell, Learning with case-injected genetic algorithms, IEEE Transactions on Evolutionary Computation 8 (4) (2004) 316–328.

[12] H. Bustince, J. Fernandez, A. Kolesárová, R. Mesiar, Generation of linear orders for intervals by means of aggregation functions, Fuzzy Sets and Systems 220 (2013) 69–77.

[13] É. Taillard, Benchmarks for basic scheduling problems, European Journal of Operational Research 64 (2) (1993) 278–285.

[14] C. Bierwirth, A generalized permutation approach to job shop scheduling with genetic algorithms, OR Spektrum 17 (1995) 87–92.

[15] A. Vargha, H. D. Delaney, A critique and improvement of the CL common language effect size statistics of McGraw and Wong, Journal of Educational and Behavioral Statistics 25 (2) (2000) 101–132.

[16] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, Swarm and Evolutionary Computation 1 (1) (2011) 3–18.

[17] M.-E. Coupvent des Graviers, L. Kobrosly, C. Guettier, T. Cazenave, Updating lower and upper bounds for the job-shop scheduling problem test instances, consolidated bounds for Taillard's ta01–ta80 (Table 14) (2025). arXiv:2504.16106.

Table 3: Mean RPD (%) restricted to the instances where the control *and* the compared arm both satisfy the plateau rule. "n both" is the size of that intersection, which differs by arm because convergence rates do. Negative Δ favours the seeded arm. This is the comparison that conditioning on the control alone cannot support, since a control-plateau instance need not be one in which the seeded arm has plateaued too.

Solver	Arm	n both	A0	Arm	Δ
GA	V2H	9	4.47	3.85	−0.62
	V2	11	4.44	4.04	−0.40
	MOR	8	3.88	5.98	+2.10
	GT	5	4.23	5.51	+1.28
	GP	9	4.27	3.64	−0.63
	MIX	10	4.14	**3.45**	**−0.69**
ABCE3	V2H	46	11.61	10.46	−1.15
	V2	46	11.61	**10.34**	**−1.27**
	MOR	43	11.69	11.95	+0.26
	GT	44	11.69	11.36	−0.34
	GP	45	11.64	10.73	−0.91
	MIX	46	11.61	10.35	−1.26
fEABCLS	V2H	27	3.54	3.53	−0.00
	V2	31	3.26	3.22	**−0.05**
	MOR	19	3.86	5.27	+1.41
	GT	27	3.49	4.11	+0.62
	GP	23	2.62	2.81	+0.20
	MIX	34	3.46	3.48	+0.02
TSN2	V2H	35	2.37	2.41	+0.04
	V2	30	2.49	2.57	+0.08
	MOR	33	2.80	3.57	+0.78
	GT	30	2.42	2.95	+0.54
	GP	34	2.43	2.78	+0.34
	MIX	35	2.32	2.34	+0.02

Table 4: Mean RPD (%) with respect to published lower bounds, over 61 instances and 30 runs per cell. Lower is better; the best arm per solver is in bold. A0 is the unseeded control.

Solver	A0	V2H	V2	MOR	GT	GP	MIX
GA	**7.52**	7.66	8.05	9.70	8.47	7.99	7.58
ABCE3	10.51	9.51	9.39	10.71	10.20	9.68	**9.37**
fEABCLS	3.67	**3.66**	**3.66**	5.16	4.47	4.04	3.67
TSN2	3.60	3.64	3.62	4.25	3.99	3.67	**3.41**

Table 5: Paired comparison of each seeded arm against the unseeded control, over 61 instances. W/NS/L: instances on which the arm is significantly better, not significantly different, or significantly worse (Wilcoxon signed-rank with Holm correction). Δ: mean signed difference in $\mathbb{E}[C_{\max}]$ (negative favours seeding). $\hat{A}_{12} < 0.5$ favours seeding.

Solver	Arm	W/NS/L	Δ	$\hat{A}_{12}$
GA	V2H	11/31/19	+4.1	0.545
	V2	9/25/27	+12.2	0.614
	MOR	1/3/57	+43.8	0.920
	GT	4/16/41	+19.9	0.739
	GP	9/21/31	+11.5	0.648
	MIX	13/29/19	+2.5	0.527
ABCE3	V2H	32/29/**0**	−21.6	0.298
	V2	**35**/26/**0**	−24.4	0.274
	MOR	1/54/6	+3.9	0.544
	GT	14/46/1	−6.1	0.436
	GP	30/31/**0**	−17.3	0.325
	MIX	**35**/26/**0**	−24.2	0.266
fEABCLS	V2H	7/43/11	+0.1	0.499
	V2	7/44/10	+0.2	0.495
	MOR	0/5/56	+28.8	0.885
	GT	3/12/46	+15.7	0.768
	GP	7/27/27	+6.8	0.640
	MIX	12/38/11	−0.4	0.499
TSN2	V2H	5/48/8	+0.6	0.518
	V2	6/47/8	−0.0	0.510
	MOR	9/13/39	+9.7	0.691
	GT	9/20/32	+5.2	0.643
	GP	10/32/19	−1.3	0.539
	MIX	**15**/41/5	−5.5	**0.436**

Table 6: Per-instance difference in RPD from the unseeded control (percentage points, negative favours seeding), with bootstrap 95% confidence intervals over instances and Holm-corrected signed-rank p-values within each solver. $n = 61$ instances.

Solver	Arm	Mean diff.	95% CI	p (Holm)	
GA	V2H	+0.141	[−0.073, +0.356]	0.358	
	V2	+0.528	[+0.240, +0.840]	0.012	*
	MOR	+2.179	[+1.887, +2.481]	< 0.001	***
	GT	+0.947	[+0.668, +1.229]	< 0.001	***
	GP	+0.467	[+0.198, +0.730]	0.002	**
	MIX	+0.055	[−0.183, +0.294]	0.546	
ABCE3	V2H	−0.997	[−1.214, −0.800]	< 0.001	***
	V2	−1.113	[−1.342, −0.899]	< 0.001	***
	MOR	+0.206	[+0.089, +0.327]	0.005	**
	GT	−0.303	[−0.492, −0.127]	0.005	**
	GP	−0.830	[−1.087, −0.596]	< 0.001	***
	MIX	**−1.140**	[−1.361, −0.930]	< 0.001	***
fEABCLS	V2H	−0.016	[−0.123, +0.092]	1.000	
	V2	−0.010	[−0.122, +0.098]	1.000	
	MOR	+1.485	[+1.244, +1.741]	< 0.001	***
	GT	+0.799	[+0.604, +0.994]	< 0.001	***
	GP	+0.366	[+0.199, +0.537]	< 0.001	***
	MIX	−0.005	[−0.130, +0.115]	1.000	
TSN2	V2H	+0.048	[−0.048, +0.143]	0.486	
	V2	+0.024	[−0.084, +0.126]	0.486	
	MOR	+0.652	[+0.393, +0.912]	< 0.001	***
	GT	+0.398	[+0.179, +0.607]	< 0.001	***
	GP	+0.070	[−0.135, +0.261]	0.456	
	MIX	**−0.187**	[−0.350, −0.045]	0.261	

Table 7: Decomposition of the MIX seeding effect on TSN2 by instance class. Δ is the mean signed change in $\mathbb{E}[C_{\max}]$ (negative favours seeding); the last column is each class's share of the total improvement.

Class	n	$\mathbb{E}[C_{\max}]$ A0	$\mathbb{E}[C_{\max}]$ MIX	Δ (%)	Share
10×10	1	937.1	944.5	+0.78	—
15×15	10	1253.6	1254.3	+0.06	—
20×20	10	1668.8	1669.1	+0.01	—
30×15	10	1844.8	1843.6	−0.06	3.6%
30×20	10	2052.6	2051.4	−0.06	3.4%
50×15	10	2780.3	2779.2	−0.04	3.2%
50×20	10	**2970.1**	**2938.4**	**−1.06**	**94.8%**

Table 8: Mean RPD (%) over all 61 instances at five fractions of the per-instance budget. Values are resampled from execution traces and agree with Table 4 to within the trace sampling resolution.

Solver	Arm	10%	25%	50%	75%	100%
GA	A0	13.58	**11.00**	**8.96**	**8.06**	**7.56**
	V2H	12.91	11.78	9.70	8.50	7.73
	V2	12.70	11.92	10.29	9.02	8.14
	MOR	14.07	12.24	10.85	10.17	9.75
	GT	13.15	11.34	9.76	8.99	8.51
	GP	12.43	11.09	9.48	8.60	8.04
	MIX	**12.10**	11.08	9.32	8.32	7.64
ABCE3	A0	12.28	11.45	10.83	10.67	10.52
	V2H	10.83	10.10	9.68	9.61	9.52
	V2	10.78	9.97	9.55	9.48	9.40
	MOR	12.84	11.95	11.11	10.92	10.73
	GT	11.84	11.12	10.52	10.35	10.22
	GP	10.65	10.17	9.84	9.76	9.68
	MIX	**10.23**	**9.76**	**9.49**	**9.46**	**9.38**
fEABCLS	A0	14.84	5.60	4.32	3.89	3.68
	V2H	9.50	5.61	4.35	3.90	**3.67**
	V2	9.36	5.63	4.35	3.90	**3.67**
	MOR	13.15	6.59	5.72	5.39	5.17
	GT	11.77	5.96	5.03	4.70	4.48
	GP	9.21	5.46	4.60	4.26	4.05
	MIX	**8.91**	**5.27**	**4.23**	**3.87**	3.67
TSN2	A0	12.48	5.98	4.36	3.79	3.53
	V2H	8.70	5.63	4.29	3.81	3.59
	V2	8.41	5.53	4.25	3.80	3.57
	MOR	10.04	5.52	4.66	4.36	4.19
	GT	9.78	5.36	4.40	4.08	3.93
	GP	7.94	5.08	4.11	3.76	3.59
	MIX	**7.89**	**5.05**	**3.94**	**3.55**	**3.36**

Table 9: Time to reach the unseeded control's final quality (+0.5%), paired by instance. Time is the fraction of the per-instance budget, and an arm that never reaches the target within the budget is recorded at 1.0. "Reach" is the percentage of the 61 instances on which the target is attained. "Earlier/later" counts the instances on which the arm reaches it before or after its own paired control, ties excluded. The sign test on those counts is the *primary* contrast, taken against the null proportion this design itself produces rather than against 0.5: since the target is drawn from the control's own endpoint, a split-half calibration under a true null (Section 7.12) puts that proportion at 0.46–0.49 depending on the solver. ΔRMTT is the paired difference in restricted mean time $E[\min(T, 1)]$, for which recording a non-reacher at 1.0 is the definition of the estimand, not an approximation to it. Both are Holm-corrected over the six seeded arms of each solver. The control reaches its own target on all 61 instances by construction, so no test is reported for it. Three representative arms per solver are shown; the full table is released with the data.

Solver	Arm	Reach	Earlier/later	p (sign)	RMTT	ΔRMTT [95% CI]
GA	A0	100%	—	—	0.62	—
	V2	56%	6/49	< 0.001	0.77	$+0.16$ $[+0.12, +0.20]$
	MOR	5%	0/59	< 0.001	0.99	$+0.37$ $[+0.31, +0.44]$
	MIX	77%	18/37	0.057	0.70	$+0.09$ $[+0.04, +0.13]$
ABCE3	A0	100%	—	—	0.38	—
	V2	100%	54/2	< 0.001	0.16	-0.22 $[-0.28, -0.16]$
	MOR	70%	13/40	0.004	0.60	$+0.22$ $[+0.14, +0.30]$
	MIX	100%	57/2	< 0.001	**0.12**	$\mathbf{-0.26}$ $[-0.31, -0.21]$
fEABCLS	A0	100%	—	—	0.44	—
	V2	89%	24/25	1.00	0.50	$+0.06$ $[+0.01, +0.11]$
	MOR	11%	0/59	< 0.001	0.94	$+0.49$ $[+0.43, +0.56]$
	MIX	89%	28/26	1.00	0.50	$+0.06$ $[+0.01, +0.12]$
TSN2	A0	100%	—	—	0.52	—
	V2	90%	35/24	0.62	0.57	$+0.05$ $[-0.00, +0.10]$
	MOR	43%	19/41	0.047	0.74	$+0.22$ $[+0.12, +0.31]$
	MIX	93%	42/15	0.001	0.46	-0.06 $[-0.11, +0.01]$

Table 10: Quality of the seeded initial population relative to the unseeded control, averaged over five instances spanning 20×20 to 50×20 (TSN2 traces). Negative is better. "Mean" is over all 250 initial individuals and is the quantity that characterises the pool.

Arm	Best individual	Mean of the 250
V2H	-11.34%	-7.35%
V2	-11.84%	-14.73%
MOR	-5.28%	-10.40%
GT	-4.75%	-8.85%
GP	$\mathbf{-12.09}\%$	$\mathbf{-15.48}\%$
MIX	-12.25%	-13.12%

Table 11: Mean Friedman ranks of the seven arms (1 = best) over 61 instances. Friedman rejects rank equality for every solver ($p < 10^{-15}$ throughout). Boldface marks arms not significantly worse than that solver's top-ranked arm (Holm-corrected paired Wilcoxon, $\alpha = 0.05$).

Solver	A0	V2H	V2	MOR	GT	GP	MIX
GA	**2.69**	**3.08**	4.25	6.41	4.77	3.85	**2.95**
ABCE3	5.60	**3.08**	**2.50**	6.14	4.96	3.31	**2.41**
fEABCLS	**2.99**	**2.93**	**3.04**	6.52	5.51	4.24	**2.78**
TSN2	3.40	3.93	3.80	5.37	5.07	3.85	**2.57**
χ^2 (df 6)	GA 132.6		ABCE3 185.0		fEABCLS 177.6		TSN2 75.5

Table 12: Mean pairwise Hamming distance of the population (0 = identical population, 1 = maximal positional difference), averaged over four instances and 10 runs. The seeded GA converges structurally *less* than its control; ABCE3 is unaffected by seeding.

Solver	Arm	Start	Mean	End	Drop
GA	A0	0.9474	0.7646	0.4639	−51.0%
	V2	0.9310	0.8845	0.7195	−22.7%
	MIX	0.9301	0.8350	0.5552	−40.3%
ABCE3	A0	0.9474	0.7140	0.8541	−9.9%
	V2	0.9310	0.7059	0.8602	−7.6%
	MIX	0.9301	0.7171	0.8497	−8.6%